Exercises Lab 2 Bonus

OTP Exercises

Questions/Exercises

1. Use one-time pad to encrypt your surname with a random key
2. Joseph and Helen have exchanged some messages using the one-time pad cryptosystem. They thought encrypting their messages with the same key is not a problem. Can you prove them wrong? Recover their messages.

Helen's ciphertext	'335855443c194a09135f10083e094059064255442c04131f125e'
Joseph's ciphertext	'2f59102c3a07561746107944370443 1c47495f117f0e5d130849101037024 059045f45162c0e'

16

1 Encrypt your surname with a random key

1.1 Mathematical Foundation

The One-Time Pad (OTP) operates on the principle of modular arithmetic:

1.1.1 Encryption Process

For each character in the plaintext:

$$C = (P + K) \mod 26 \tag{1}$$

Where:

- P: Plaintext character value (A=0, B=1, ..., Z=25)
- K: Key character value (A=0, B=1, ..., Z=25)
- C: Ciphertext character value (A=0, B=1, ..., Z=25)

1.1.2 Decryption Process

$$P = (C - K) \mod 26 \tag{2}$$

1.2 Implementation

```python
import secrets

def one_time_pad_encode(plaintext):
    plaintext = ''.join(c for c in plaintext.upper() if c.isalpha())
    # Generate random key of same length
    key = ''.join(chr(secrets.randbelow(26) + 65) for _ in range(len(plaintext)))

    # Encrypt using modular addition
    ciphertext = ''
    for p_char, k_char in zip(plaintext, key):
        p_val = ord(p_char) - 65
        k_val = ord(k_char) - 65
        c_val = (p_val + k_val) % 26
        ciphertext += chr(c_val + 65)

    return ciphertext, key

# Running commands

surname = "Avduli"
cipher, key = one_time_pad_encode(surname)

print(f"Plaintext:  {surname}")
print(f"Key:        {key}")
print(f"Ciphertext: {cipher}")
```

1.3 Output

```
Plaintext:  Avduli
Key:        SSULXU
Ciphertext: SNXFIC
```

2 Joseph's and Helen's Key

2.1 Vulnerability Overview

When the same one-time pad key is reused to encrypt multiple messages, the cryptosystem becomes vulnerable to a **known-plaintext attack** through XOR analysis. Given:

$$c_1 = m_1 \oplus k$$
$$c_2 = m_2 \oplus k$$

where c_1, c_2 are ciphertexts and m_1, m_2 are plaintexts encrypted with the same key k.

2.2 Key Mathematical Insight

XORing the two ciphertexts eliminates the key:

$$\begin{aligned} c_1 \oplus c_2 &= (m_1 \oplus k) \oplus (m_2 \oplus k) \\ &= m_1 \oplus m_2 \oplus (k \oplus k) \\ &= m_1 \oplus m_2 \end{aligned}$$

Thus we obtain the XOR of the original plaintexts without knowledge of the key.

2.3 Crib Dragging Technique

The attack proceeds using **crib dragging**:

2.3.1 Step 1: XOR Ciphertexts

```
Helen's ciphertext (c1): 335855443c194a09135f10083e094059064255442c...
Joseph's ciphertext (c2): 2f59102c3a0756174610794370443lc47495f117...
XOR result (m1 xor m2): 1c014568061e1c1e554f694c090d03410b0a554e0c1a17...
```

2.3.2 Step 2: Assume Known Plaintext Fragments

We guess common English words or phrases ("cribs") and test them at all positions:

For a crib g at position i:

$$m_2[i : i + \text{len}(g)] = (m_1 \oplus m_2)[i : i + \text{len}(g)] \oplus g$$
$$m_1[i : i + \text{len}(g)] = (m_1 \oplus m_2)[i : i + \text{len}(g)] \oplus m_2[i : i + \text{len}(g)]$$

2.3.3 Step 3: Validate Results

If the resulting text in the other message is also readable English, we have found a valid crib. Common successful cribs include:

- `" the "`, `" and "`, `" to "` (most common English trigrams)
- `"Hello "`, `"Hi "` (common message starters)
- `"Joseph"`, `"Helen"` (known names from context)

2.4 Message Recovery Process

1. **Identify message boundaries**: Look for spaces, punctuation, and common words
2. **Propagate constraints**: Each recovered character constrains the corresponding character in the other message
3. **Iterative refinement**: Use frequency analysis and context clues to resolve ambiguities
4. **Cross-verification**: Ensure both messages remain coherent as more text is recovered

2.5 Recovered Messages

After applying crib dragging and frequency analysis, the plaintexts were recovered:

```
b'The crypto labs are so fun'
b'Hi Helen! I hope you enjoy this course'
```

2.6 Implementation

For crib, let us assume that one of them greets the other, so let us say, Helen says "Hi Joseph" and Joseph says "Hi Helen". We xor those keys and see the rseults

```python
from pwn import xor

c1 = bytes.fromhex('335855443c194a09135f10083e094059064255442c04131f125e')
c2 = bytes.fromhex('2f59102c3a0756174610794437044431c47495f117f0e5d13084910103702405904 5f45162c0e')


crib = b'Hi Helen'
crib2= b'Hi Joseph'
print(xor(crib, c1))
print(xor(crib, c2))

print(xor(crib2, c1))
print(xor(crib2, c2))
```

```
b'{1u\x0cYu/g[60@[e%7N+u\x0cIhvqZ7'
b'g00d_k3y\x0eyY\x0cRh&r\x0f \x7fY\x1ab8}@ 0XRn%7L6e^Ib'
b'{1u\x0eSj/y{\x17y(tf3<v*\x1d-\x0cN|lw.'
b'g00fUt3g.X\x10d}k0y7!\x17x_D2`m9xX^"\n6w:5~dg'
```

we can notice a key but we don't know yet if it is the key, let's make the crib a little longer and add "! I" to each and see the results

```
b"{1u\x0cYu/g2\x7fY@W)\x08<j';e\x0cM[v2\x16"
b'g00d_k3yg00\x0c^$\x0by+,10_G\x15z(\x01u|RlayM\x17,6dk'
b'{1u\x0eSj/y{~0Av``\x13i104D%3VZ7'
b'g00fUt3g.1Y\r\x7fmcV(::a\x17/}Z@ 0ZXq%)l~e_dg'
```

the key seems to repeat itself, so let's xor the messages with it and see the result

```
1  key=b'g00d_k3y'
2  print(xor(c1,key))
3  print(xor(c2,key))
```

```
b'The crypto labs are so fun'
b'Hi Helen! I hope you enjoy this course'
```